

Java Programming 3

Week 11: Exception Handling



Agenda this week

Default Exception Handling in Spring

Add `@ResponseStatus`

Controller Based

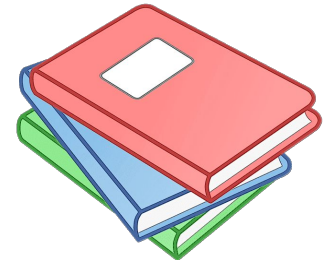
Global Exception Handling

What to use when?

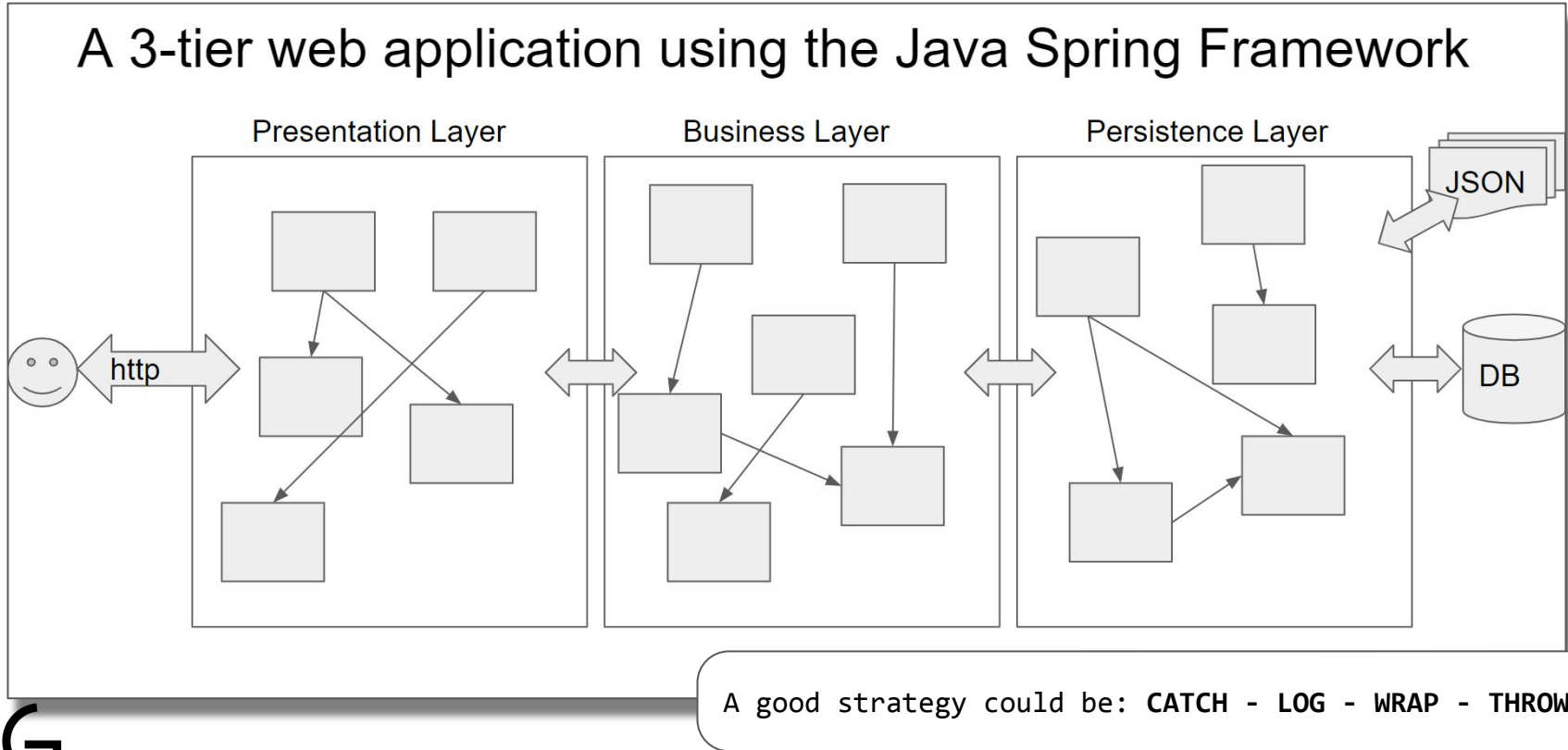
Final project release - Oral exam

Tutorials

- <https://www.baeldung.com/spring-boot-custom-error-page>
- <https://spring.io/blog/2013/11/01/exception-handling-in-spring-mvc>



General thoughts on Exception Handling in 3-tier



Demo: Flightbooking application



- Let's have a look at a small flightbooking application
- <https://gitlab.com/kdg-ti/programming-3/exercises/flightbooking>
 - A user can book a flight
 - Booking a flight will result in inserting records in 2 tables
 - PASSENGER_INFO: contains information about the booked flight
 - PAYMENT_INFO: contains information about the payment.
 - In the service layer we will use @Transactional to ensure that the complete transaction is committed or rolled back in case of problems.



Exception handling in Spring?

- If we look at the Flightbooking demo, several problems can occur:
 - The Service layer can throw these custom exceptions:
 - `DestinationNotAvailableException`: when we try to book a flight to a non-existing destination
 - `InsufficientMoneyException`: when we try to book a flight but we cannot pay for it...
 - `InvalidAccountException`: when the account is not a valid one.
 - What happens in the Repository layer if the database fails?
 - What happens if we try to surf to a non-existing path on the server?
- Can we handle these problems in a *uniform* way?





Agenda this week

Default Exception Handling in Spring

Add @ResponseStatus

Controller Based

Global Exception Handling

What to use when?

Final project release - Oral exam

Spring Boot and exceptions

- Spring boot (with Thymeleaf) looks for a /error.html page in the templates
- No /error.html → Spring Boot uses the “Whitelabel Error Page”

Whitelabel Error Page

This application has no explicit mapping for /error, so you are seeing this as a fallback.

Mon Dec 13 00:41:48 CET 2021

There was an unexpected error (type=Bad Request, status=400).

You can disable the Whitelabel Error Page with this setting in application.properties:
`server.error.whitelabel.enabled=false`
(but that is even worse)

If you create your own Thymeleaf error page, the Error pages generated by Spring Boot's error configuration receive `${timestamp}`, `${status}`, `${error}`, `${message}` as variables. You can use them in your custom error page!

HTTP Status codes?

- HTTP Response contains status code on the first line:

```
HTTP/1.1 500
```

```
Content-Type: text/html; charset=UTF-8
```

```
Content-Language: en-US
```

```
Content-Length: 287
```

```
Date: Thu, 22 Dec 2022 23:09:23 GMT
```

```
Connection: close
```

Inspect this using the developer tools of your browser...!

HTTP Status codes?

- 1xx informational response – the request was received, continuing process
- 2xx successful – the request was successfully received, understood, and accepted
- 3xx redirection – further action needs to be taken in order to complete the request
- 4xx client error – the request contains bad syntax or cannot be fulfilled
- 5xx server error – the server failed to fulfil an apparently valid request

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>

Different page per status code

- Specific page per status code is possible:
 - Save under templates/error/
 - Give it name of status code
 - Eg templates/error/500.html

Or, you can write your own `ExceptionHandler` to show an error page depending on the status code.

See <https://www.baeldung.com/spring-boot-custom-error-page> for details...

Exercise 1:

- Download the Flightbooking demo
- Inspect the errorpages it shows when the different possible errors occur:
 - Not enough money - Account not valid - Destination not available
 - Database error (drop a table using the h2-console and book flight afterwards)
 - Surf to not mapped path
 - ...
- Replace the errorpage by a custom error page, use the `${error}` and `${status}` variables...
- Add a custom 500.html page under /error: test!





Agenda this week

Default Exception Handling in Spring

Add @ResponseStatus

Controller Based

Global Exception Handling

What to use when?

Final project release - Oral exam

@ResponseStatus

- Any unhandled exception will cause the server to respond with a HTTP 500 response
- Exception you write yourself you can annotate with @ResponseStatus to change that

```
@ResponseStatus (value=HttpStatus.NOT_FOUND, reason="No such Place") //  
404  
public class DestinationNotAvailableException extends RuntimeException {  
    // ...  
}
```

If you do not specify a reason, the exception message is used.

Exercise 2:

- Add response status codes to
 - InsufficientMoneyException: 400 (BAD REQUEST)
 - DestinationNotAvailable: 404 (NOT FOUND)
- Check the result
 - Use the browsers inspect tool to check the HTTP status code!





Agenda this week

Default Exception Handling in Spring

Add @ResponseStatus

Controller Based

Global Exception Handling

What to use when?

Final project release - Oral exam

Controller Based exception handling

- You can write try catch blocks in the controller methods
- Direct the user to custom error view

```
@PostMapping("/bookflight")
public String bookFlight(FlightBookingDTO flightBookingDTO){
    try {
        flightBookingService.bookFlight(flightBookingDTO.getName(),
                                         flightBookingDTO.getDestination(), flightBookingDTO.getAccount());
    } catch (DestinationNotAvailableException e) {
        log.error(e.getMessage());
        return "error";
    }
    return "redirect:index";
}
```

Exception handling code in same method as code for the normal flow, can we separate?

Controller Based exception handling

- You can add `@ExceptionHandler` annotated methods to your Controller
 - Add the exception classes as values to this annotation
- They will handle the exceptions thrown by your `@RequestMapping` methods
- You can
 - Handle exceptions that don't have a `@ResponseStatus` annotation
 - Redirect the user to a dedicated error view
 - Build a totally custom error response

```
@ExceptionHandler({InvalidAccountException.class})
public String handleInvalidAccountException(Exception e, Model model){
    log.error(e.getMessage());
    model.addAttribute("error", "Bad account");
    return "error";
}
```

Controller Based exception handling

```
// Convert a predefined exception to an HTTP Status code
@ResponseStatus(value=HttpStatus.CONFLICT,
                reason="Data integrity violation") // 409
@ExceptionHandler(DataIntegrityViolationException.class)
public void conflict() {
    // Nothing to do
}
```

You can use it also just to change the HTTP Response Status for a specific exception...

```
@ExceptionHandler({SQLException.class, DataAccessException.class})
public String databaseError() {
    return "databaseError";
}
```

You can use it also just to return a different view for a specific exception...

Exercise 3:

- Add a try catch block in the bookflight method for the `DestinationNotAvailableException` and redirect to error view
- Add exception handling methods to the Controller for the 2 other exceptions and redirect to error view, pass on the error message to the view using a Model.
- Log the exception using the logging framework





Agenda this week

Default Exception Handling in Spring

Add @ResponseStatus

Controller Based

Global Exception Handling

What to use when?

Final project release - Oral exam

Global Exception Handling

- Using `@ControllerAdvice` classes
 - Same technique as for Controller
 - But across the whole application → all controllers
 - Supports 3 types of methods:
 - `@ExceptionHandler`
 - `@ModelAttribute`
 - `@InitBinder`
- We only look at `@ExceptionHandler`

Global Exception Handling: example

```
@ControllerAdvice
class GlobalExceptionHandler {
    @ResponseStatus(HttpStatus.CONFLICT)    // 409
    @ExceptionHandler(DataIntegrityViolationException.class)
    public void handleConflict() {
        // Nothing to do
    }
}
```

In the example above, all `DataIntegrityViolationExceptions` will be handled this way across the complete application (all Controllers)

Global Exception Handling: one method for all exceptions?

```
@ControllerAdvice
class GlobalExceptionHandler {
    @ExceptionHandler(Exception.class)
    public ModelAndView
    defaultErrorHandler(HttpServletRequest req, Exception e) throws Exception {
        if (AnnotationUtils.findAnnotation
            (e.getClass(), ResponseStatus.class) != null) {
            throw e;
        }
        // Otherwise setup and send the user to a default error-view.
        ModelAndView mav = new ModelAndView();
        mav.addObject("exception", e);
        mav.addObject("url", req.getRequestURL());
        mav.setViewName(DEFAULT_ERROR_VIEW);
        return mav;
    }
}
```

If the exception is already annotated, we rethrow it and let the Spring framework handle it. Otherwise we handle it here...

Exercise 4:

- Add a global exception handling class using the `@ControllerAdvice` annotation
- Add a method there to handle all `DataAccessExceptions` in a uniform way...





Agenda this week

Default Exception Handling in Spring

Add @ResponseStatus

Controller Based

Global Exception Handling

What to use when?

Final project release - Oral exam

What to use when?

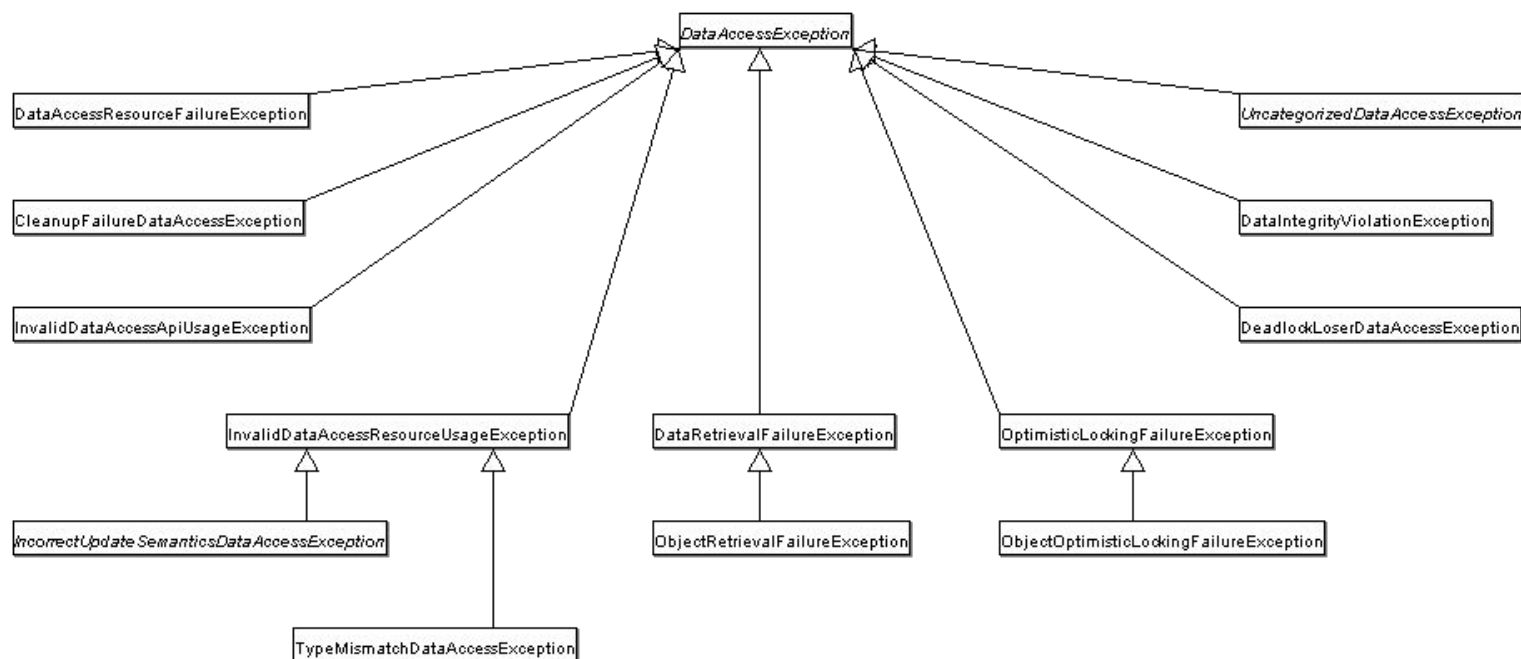
- For exceptions you write, consider adding `@ResponseStatus` to them.
- For all other exceptions implement an `@ExceptionHandler` method on a `@ControllerAdvice` class
- For Controller specific exception handling add `@ExceptionHandler` methods to your controller.

Warning: Be careful mixing too many of these options in the same application. If the same exception can be handled in more than one way, you may not get the behavior you wanted. `@ExceptionHandler` methods on the Controller are always selected before those on any `@ControllerAdvice` instance. If you have multiple `@ControllerAdvice` instances you can define the processing order by annotating them with `@Order`.

Exceptions in jparepository

- Spring wraps the JDBC exceptions in unchecked exceptions:

<https://docs.spring.io/spring-framework/docs/current/javadoc-api/org/springframework/dao/DataAccessException.html> ,



Agenda this week

Default Exception Handling in Spring
Add @ResponseStatus
Controller Based
Global Exception Handling
What to use when?
Final project release - Oral exam

Project

- Add exception handling to your project:
 - Provide two custom error pages:
 - One page is used for all database problems
 - One page is used for all other problems
 - Implement a custom Exception for your application (extends RuntimeException)
 - Work out a situation in your service layer that can throw this custom exception
 - Use a `@ControllerAdvice` class to handle all database exceptions
 - Handle the custom Exception at the Controller level
 - Be sure to log all exceptions using the logging framework



Oral Exam

- **You cannot participate in the exam without a project!**
- Be sure to have your project running on your laptop when you enter the exam!
 - You give a **short demo (max 5 minutes)**
- You will be questioned on the ENTIRE course material (all content covered in the slides)
- Questioning will be done *using* your project:
 - Explain code
 - **Modify code**, add new functionality → **live coding!**
 - Explain concepts



Evaluation

- Your project must be fully developed, as this demonstrates that you have mastered the course material.
- **If certain assignments are not implemented, you cannot prove your understanding of those topics and cannot score points for them.**
- For example:
 - If you have implemented only 9 of the 12 assignments (75%), your MAXIMUM score for the exam will be 15 out of 20.
 - If you have only implemented half of the assignments, your MAXIMUM score will be 10 out of 20.
- Final evaluation will be a grade based on the answers to the questions combined with the completeness of your project

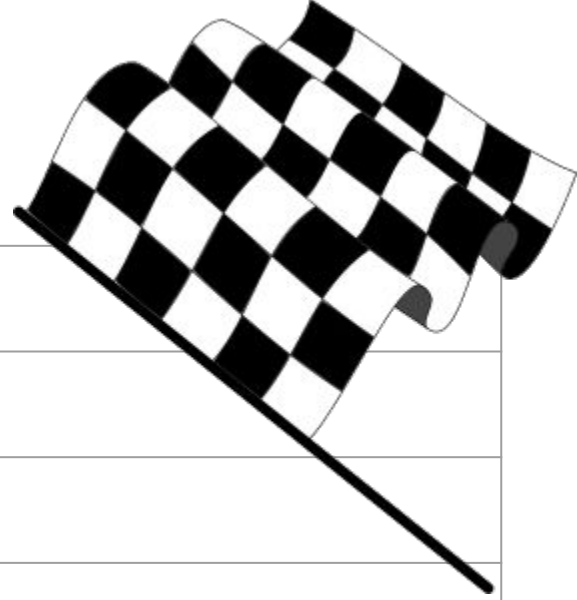


The project - final release

- Add a README.MD file to the root of the project
- In this file:
 - Your name
 - Explanation of your domain and the relations between the entities (high level explanation)
 - Explanation of your profiles
 - What database do you use, the connection parameters
 - Any extra information needed to be able to run the project smoothly
 - start URL for web application
- Be sure to...
 - clean up the code (remove obsolete/commented out code...)
 - add comments if necessary
- Push to gitlab - tag as “final” - deadline: **01-01-2026 23:59**
 - Add teacher as Reporter to your repository



Agenda this week



Default Exception Handling in Spring
Add @ResponseStatus
Controller Based
Global Exception Handling
What to use when?
Final project release - Oral exam